

实验作业 4

实验作业 4

一、实验任务

任务一至三

任务四：Conditional GAN

二、作业提交

三、参考文献及资料

本实验旨在让同学初步理解生成对抗网络（Generative Adversarial Network, GAN）的基本原理与实现方法。通过实现一个最基本的 GAN 模型的搭建与训练，掌握生成器与判别器的对抗训练机制，并在此基础上，进一步学习 DCGAN、WGAN 和 Conditional GAN 等 GAN 的典型变体。

一、实验任务

本次实验包含 4 个任务，分数占比如下：

- 任务一：Basic GAN (20%)
- 任务二：DCGAN (20%)
- 任务三：WGAN-GP (20%)
- 任务四：Conditional GAN (40%)

从知识上来说，任务一和任务二是后续任务的基础，建议按照先后顺序依次完成。

任务一至三

在任务一、二、三中，已为同学们提供相应的 Jupyter Notebook 文件，对应关系如下：

- 任务一：Task1/Basic-GAN.ipynb
- 任务二：Task2/DCGAN.ipynb
- 任务三：Task3/WGAN-GP.ipynb

其中**任务一**引导同学逐步实现并训练一个最基本的生成对抗网络（GAN），理解生成器（Generator）与判别器（Discriminator）的结构及其对抗训练过程。

任务二介绍了传统 GAN 的一种改进版本 DCGAN（Deep Convolutional Generative Adversarial Network，深度卷积生成对抗网络），其核心思想是：用卷积神经网络（CNN）替代全连接网络，让生成器（Generator）和判别器（Discriminator）都具备从图像中自动学习空间特征的能力，从而在生成图像时更加稳定、真实。

任务三则引导大家学习一种训练时稳定性更高的 GAN 变体，即 WGAN-GP，并了解其稳定性的来源：**Wasserstein 损失函数**和**梯度惩罚项（Gradient Penalty）**。

最终提交的 `.ipynb` 文件需要保证所有提供的测试代码均可通过且训练过程能正常进行。

任务四：Conditional GAN

条件化生成对抗网络（Conditional GAN），顾名思义，是在基础的 GAN 框架上引入条件变量（Conditional Variable），以实现有监督的、可控的样本生成。

Conditional GAN 在现实中比较常见的应用场景包括根据动物物种名生成对应图片，或者根据人外观的描述生成符合条件的人脸图片等。而以我们一直使用的 MNIST 数据集为例，我们可以控制 GAN 生成 0 ~ 9 中的哪个手写数字。

在任务四中，需要你根据之前的经验，实现一个完整的、相对简单的 Conditional GAN，并尝试训练之。

下面提供一个可供参考的实现思路：

- `Generator` 和 `Discriminator` 可以直接使用你在任务二中实现的。
- 用于获取条件变量的函数 `get_one_hot_labels`，即一个条件变量由一个长度为 10 的一维张量表示，分别代表手写数字 0 ~ 9。

```
1 import torch.nn.functional as F
2 def get_one_hot_labels(labels, n_classes):
3     '''
4     Function for creating one-hot vectors for the labels, returns a tensor of shape (?,
5     num_classes).
6     Parameters:
7         labels: tensor of labels from the dataloader, size (?)
8         n_classes: the total number of classes in the dataset, an integer scalar
9     '''
10    # TODO
11    pass
```

下面是对该函数的简单测试脚本，方便同学们判断正确性：

```
1 assert (
2     get_one_hot_labels(
3         labels=torch.Tensor([[0, 2, 1]]).long(),
4         n_classes=3
5     ).tolist() ==
6     [[
7         [1, 0, 0],
8         [0, 0, 1],
9         [0, 1, 0]
10    ]]
11 )
12 print("Success!")
```

- 用于连接 noise 和条件变量的函数 `combine_vectors`。

```

1  def combine_vectors(x, y):
2      '''
3      Function for combining two vectors with shapes (n_samples, ?) and (n_samples, ?).
4      Parameters:
5          x: (n_samples, ?) the first vector.
6              In this assignment, this will be the noise vector of shape (n_samples, z_dim),
7              but you shouldn't need to know the second dimension's size.
8          y: (n_samples, ?) the second vector.
9              Once again, in this assignment this will be the one-hot class vector
10             with the shape (n_samples, n_classes), but you shouldn't assume this in your code.
11      '''
12      # Note: Make sure this function outputs a float no matter what inputs it receives
13      # TODO
14      return combined

```

测试脚本如下：

```

1  mbined = combine_vectors(torch.tensor([[1, 2], [3, 4]]), torch.tensor([[5, 6], [7, 8]]))
2  # Check exact order of elements
3  assert torch.all(combined == torch.tensor([[1, 2, 5, 6], [3, 4, 7, 8]]))
4  # Tests that items are of float type
5  assert (type(combined[0][0].item()) == float)
6  # Check shapes
7  combined = combine_vectors(torch.randn(1, 4, 5), torch.randn(1, 8, 5))
8  assert tuple(combined.shape) == (1, 12, 5)
9  assert tuple(combine_vectors(torch.randn(1, 10, 12).long(), torch.randn(1, 20,
10  12).long()).shape) == (1, 30, 12)
11 print("Success!")

```

- 仿照任务一中的训练流程尝试写出 Conditional GAN 的训练脚本（这时候 Generator 的输入长度是多少？Discriminator 呢？）

二、作业提交

本次作业采取个人提交的方式，请各位同学务必自主独立完成，一旦发现学术不端行为，本次实验成绩作废。

提交格式：

- 保留作业文件夹原有目录结构，在此基础上：
 - 补全任务一、二、三中要求的代码实现，保证能通过给定的测试并能正常训练。
 - 新建目录 `Task4`，任务四的代码放在其中，可以提供简要的 README 来对代码进行说明。
- 将以上内容打成压缩包提交，压缩包命名为“学号-姓名-实验作业4”。

作业截止提交日期：12 月 15 日。

三、参考文献及资料

[1] Radford A, Metz L, Chintala S. *Unsupervised representation learning with deep convolutional generative adversarial networks*[J]. arXiv preprint arXiv:1511.06434, 2015.

[2] PyTorch API 手册: <https://docs.pytorch.org/docs/stable/pytorch-api.html>